Communication Protocols Interview Preparation - Complete

UART, SPI, I2C & CAN — Questions & Answers

Sriram M

UART

Q1. What is UART? Universal Asynchronous Receiver/Transmitter — a serial communication protocol/peripheral that transmits data one bit at a time over a single wire (TX) with no shared clock, using start/stop bits to frame each byte and a pre-agreed baud rate for timing.

Q2. Why "asynchronous"? Because there's no clock line shared between transmitter and receiver — both sides must independently run at the same configured baud rate, and the receiver synchronizes to each new byte using the start bit's falling edge.

Q3. UART frame format? Idle line is high (1). A frame consists of: 1 start bit (always 0), 5-9 data bits (usually 8), an optional parity bit, and 1-2 stop bits (always 1) that return the line to idle.

Q4. Baud rate? The number of bits transmitted per second (e.g., 9600, 115200). Both transmitter and receiver must be configured to the same baud rate (within a small tolerance, typically <2-3%), or bits get misread as the sampling point drifts.

Q5. Parity bit? An optional extra bit used for basic error detection — even parity makes the total number of 1-bits (including the parity bit) even; odd parity makes it odd. The receiver recomputes parity and flags a parity error on mismatch, though it can't correct errors or detect an even number of bit-flips.

Q6. Start bit and stop bit? The start bit (always dominant/0) signals the beginning of a new frame and lets the receiver synchronize its bit-sampling clock; the stop bit(s) (always 1) mark the end of the frame and guarantee the line returns to idle before the next frame.

Q7. Full duplex vs half duplex? Full duplex means both devices can transmit and receive simultaneously (separate TX/RX wires, as standard UART does); half duplex means only one direction can be active at a time on a shared line (e.g., RS-485 in 2-wire mode), requiring direction control/turnaround time.

Q8. UART vs USART? USART (Universal Synchronous/Asynchronous Receiver/Transmitter) supports everything UART does plus an optional synchronous mode with a shared clock line, enabling higher/more precise data rates and interfacing with synchronous peripherals; plain UART only supports asynchronous mode.

Q9. Overrun error? Occurs when a new byte is fully received before the CPU/software has read the previous byte out of the receive data register (or buffer), causing the earlier unread byte to be overwritten/lost. Usually indicates the receiver isn't servicing the UART fast enough (e.g., missed interrupt, blocked ISR).

Q10. Framing error? Detected when the receiver doesn't see the expected stop bit(s) at the correct bit position (finds a 0 where a 1 was expected) — usually caused by a baud rate mismatch between

transmitter and receiver, or noise corrupting the timing.

Q11. UART FIFO? A small hardware buffer (e.g., 16 bytes) inside the UART peripheral that queues incoming/outgoing bytes, reducing the interrupt rate (CPU can be interrupted once per several bytes instead of every byte) and providing headroom against brief CPU servicing delays.

Q12. Interrupt-driven UART? The UART peripheral raises an interrupt when a byte is received (RX) or the transmit buffer is empty (TX), and the ISR handles reading/writing data (often into a ring buffer) without the CPU having to poll status registers continuously.

Q13. DMA with UART? Configuring a DMA channel to automatically move received bytes from the UART data register into a memory buffer (or transmit from memory to the UART) without CPU intervention per byte — useful for high-throughput or fixed-length message transfers, freeing the CPU for other work.

Q14. RS-232 vs UART logic level? UART peripherals operate at logic/TTL levels (0V/3.3V or 0V/5V); RS-232 is a physical-layer standard using higher voltage swings (e.g., ±3V to ±15V) and inverted logic, requiring a level-shifter IC (e.g., MAX232) to convert between UART and RS-232 for longer-distance/noisier links.

Q15. RS-485? A differential, multi-drop serial standard often layered over UART framing, using two wires (A/B) for noise-resistant, longer-distance (up to ~1200m), half-duplex communication among multiple nodes — commonly used in industrial and automotive sub-networks.

Q16. Auto baud detection? A technique where the receiver measures the timing of the start bit (or a known calibration character) of the first incoming frame to automatically determine and configure the correct baud rate, useful when the baud rate isn't known in advance (e.g., bootloaders).

Q17. Break condition? A UART signal held at the space (0/low) level for longer than a full character frame (including stop bits) — used to signal special conditions like a reset request or to gain the receiver's attention outside normal framing, since it violates normal stop-bit timing.

Q18. Bootloader over UART? A common embedded pattern where a small resident bootloader program communicates over UART (using a simple protocol like Xmodem/Ymodem or a custom command set) to receive new firmware images and write them to flash, enabling field firmware updates without a JTAG/SWD debugger.

Q19. Common UART bugs/issues? Baud rate mismatch, missing/incorrect ground reference between devices, buffer overrun from slow ISR servicing, wrong parity/stop-bit configuration, floating RX line causing noise/garbage bytes, and TX/RX lines swapped between two devices.

Q20. UART debugging approach? Verify both ends use identical baud rate/parity/stop bits/frame format, check TX-to-RX (crossed) wiring and common ground, use a logic analyzer or oscilloscope to capture actual bit timing/levels, and check for overrun/framing error flags in the UART status register during data loss.

Advanced

Q21. Flow control (RTS/CTS)? Hardware flow control uses two extra lines — RTS (Request To Send) and CTS (Clear To Send) — so a receiver can signal the transmitter to pause when its buffer is nearly full, preventing overrun on slower or busy receivers; software flow control achieves the same with in-band XON/XOFF characters instead of extra wires.

Q22. Multi-drop UART? A configuration (often over RS-485) where multiple devices share a single UART bus with addressing built into the higher-level protocol (since UART itself has no addressing), and only one transmitter drives the line at a time, coordinated by a master or bus arbitration scheme.

Q23. Oversampling in UART receivers? The UART peripheral samples each bit multiple times (commonly 16x the baud rate) and uses a majority/mid-point vote to determine the bit value, improving noise immunity and tolerance to small clock/baud-rate mismatches between transmitter and receiver.

Q24. Baud rate error tolerance? Because there's no shared clock, small differences between the transmitter's and receiver's actual baud rate accumulate bit-by-bit across a frame; if the cumulative timing drift exceeds roughly half a bit period by the last bit, that bit is misread. Practically, total baud rate error should stay under about 2-3% for reliable 8N1 communication.

Q25. IrDA / other UART-derived protocols? IrDA is an infrared, UART-framed protocol (using pulse-based encoding instead of direct voltage levels) for short-range wireless links; other protocols like LIN

and single-wire debug UART reuse UART's asynchronous framing over different physical layers.

Q26. Glitch filtering on UART RX? Some UART peripherals implement a digital noise/glitch filter on the RX line, rejecting sub-bit-width spurious pulses (from EMI or contact bounce) before they're interpreted as a false start bit — reduces spurious wake-ups or corrupted frames in electrically noisy environments.

Q27. UART in bootloaders — protocol design? A bootloader UART protocol typically defines a simple command/ACK scheme (e.g., sync byte, command ID, length, payload, checksum) so the host tool can reliably detect frame boundaries and verify integrity, since raw UART alone provides no message framing beyond individual bytes.

Q28. Synchronization overhead? Each UART byte carries at least 2 bits of overhead (1 start + 1 stop, more with parity/2 stop bits) on top of the payload — for an 8N1 frame that's 10 bits per byte, meaning only 80% of the raw bit rate is usable data throughput.

Q29. UART over USB (CDC/virtual COM port)? A USB CDC-ACM device presents itself to the host OS as a standard virtual serial (COM) port, tunneling the actual byte stream over USB packets — useful for connecting a modern PC without a physical UART/RS-232 port using the same software driver model.

Q30. Voltage level mismatches (3.3V vs 5V)? Directly connecting a 5V UART TX to a 3.3V-only RX can exceed the receiver's absolute maximum voltage rating and damage it; a level shifter (resistor divider for TX-only-5V-to-3.3V, or a proper bidirectional level-shifter IC) should be used to safely interface mismatched logic levels.

Scenario-Based

Q31. Receiving garbage/corrupted characters. Almost always a baud rate mismatch (including if one side uses a slightly off crystal/clock), wrong parity/stop-bit configuration, or a floating/noisy RX line — verify configuration matches exactly on both ends and check signal integrity with a scope.

Q32. Works at low baud rate but fails at high baud rate. Points to timing margin issues — long/high-capacitance wiring, insufficient drive strength, an inaccurate clock source, or the receiver's ISR/DMA not servicing the FIFO fast enough at the higher data rate, causing overruns.

Q33. First byte after idle is corrupted, rest is fine. Often caused by the receiver's UART not being fully synchronized/settled after a long idle period, a false start-bit glitch waking the receiver early, or the transmitter's first byte being sent before the line has properly stabilized (e.g., right after power-up or mode change).

Q34. Data flows one direction but not the other. Check that TX/RX aren't swapped only in one direction, confirm both devices actually share a common ground, verify the non-working side's UART is enabled/configured, and check for a hardware fault or incorrect pin-mux on that specific line.

Q35. Intermittent data loss under load. Usually an overrun error from the CPU not reading the RX FIFO fast enough (blocked by a higher-priority ISR or long critical section) — check for overrun status flags, consider enabling DMA or increasing FIFO threshold/interrupt priority.

Q36. Checksum/CRC mismatches at the application layer. Even with correct UART framing, if a custom protocol's checksum fails intermittently, look for byte loss/duplication from overruns, buffer race conditions between ISR and main loop, or off-by-one errors in frame parsing rather than the electrical link itself.

Q37. Bootloader not responding over UART. Check that the target's boot-mode pin/strap is correctly set to enter bootloader mode, confirm baud rate and framing match the bootloader's fixed configuration (bootloaders often don't support auto-detect), and verify the reset/entry sequence timing expected by the specific bootloader.

Q38. Only garbage/no signal at all is seen on the scope. Confirm the UART peripheral is actually enabled and the correct pins are mux'd to the UART function (not left as GPIO or another peripheral), and check the baud rate divider register is set correctly for the actual peripheral clock frequency.

Q39. Ground loop / noise on long UART cable runs. Long cables sharing a poor ground reference between devices can pick up induced noise and shift the effective logic thresholds — solutions include using a differential standard like RS-485 instead of single-ended UART, adding ground isolation, or shortening/shielding the cable.

Q40. Debugging a suspected UART timing/protocol issue. Use a logic analyzer to capture raw bit timing and decode the frame at the electrical level (compare actual vs configured baud rate), check

status/error registers (overrun, framing, parity) on both transmitter and receiver, and isolate whether the fault is electrical (scope) or software (ISR/buffer logic).

SPI

Q21. What is SPI? Serial Peripheral Interface — a synchronous, full-duplex, master-slave serial protocol using a shared clock (SCLK) plus separate MOSI/MISO data lines and per-slave chip-select (CS) lines, commonly used for high-speed communication with sensors, flash memory, and displays.

Q22. SPI signal lines? SCLK (clock, driven by master), **MOSI** (Master Out Slave In), **MISO** (Master In Slave Out), and **CS/SS** (Chip Select/Slave Select, active-low, one per slave) to select which device is being addressed.

Q23. SPI modes (CPOL/CPHA)? Defined by two configuration bits: **CPOL** (clock idle state — 0 = idle low, 1 = idle high) and **CPHA** (which clock edge data is sampled on — 0 = sampled on the first/leading edge, 1 = on the second/trailing edge), giving four modes (0-3). Master and slave must use matching mode settings.

Q24. Master-slave relationship? The master generates the clock and initiates all transactions (selecting a slave via CS and driving data on MOSI); slaves only respond/shift out data on MISO when their CS is asserted, and never initiate communication on their own.

Q25. Multiple slave configuration? Typically each slave has its own dedicated CS line from the master (independent-slave/parallel configuration), so only one CS is asserted at a time; alternatively, a “daisy-chain” configuration shifts data through multiple slaves in series using one shared CS.

Q26. SPI vs I2C? SPI is full-duplex with separate MOSI/MISO lines, generally faster, but needs more pins (one CS per slave) and has no built-in addressing or acknowledgment. I2C uses only 2 shared wires (SDA/SCL) with device addressing and ACK, supports more devices with less wiring, but is half-duplex and typically slower.

Q27. Clock polarity and phase? CPOL sets whether the clock line idles high or low between transactions; CPHA sets whether data is captured on the first or second clock transition after CS assertion — together they determine exactly when data must be valid/stable relative to the clock edges.

Q28. Daisy chaining in SPI? Connecting the MISO of one slave to the MOSI of the next, forming a shift-register-like chain across a shared clock and single CS — the master shifts data through the whole chain in sequence, common in cascaded shift registers/LED drivers.

Q29. Full duplex operation? SPI transmits and receives data simultaneously on each clock edge — every clock pulse shifts one bit out on MOSI and one bit in on MISO at the same time, so a “read” naturally happens during every “write” transaction (and vice versa).

Q30. SPI speed? Determined by the SCLK frequency, which can range from a few hundred kHz up to tens of MHz depending on the MCU peripheral, wiring length/quality, and the slave device’s maximum rated clock speed — much faster than typical I2C or UART.

Q31. Chip select handling? The master must assert (drive low, for active-low CS) the target slave’s CS line before starting the clock/data transfer and hold it asserted for the entire transaction, then de-assert it afterward — incorrect CS timing is a very common cause of failed SPI transactions.

Q32. SPI with DMA? Configuring DMA to automatically transfer a block of TX data to the SPI data register and/or move received RX data to a memory buffer without per-byte CPU intervention — important for high-speed/large transfers like reading sensor FIFOs or writing to displays/flash.

Q33. SPI flash interfacing? Communicating with SPI NOR/NAND flash chips using standard commands (read ID, read/write/erase sector, status register poll) sent over MOSI while clocking data in/out on MISO, respecting the flash’s specific command opcodes and timing/erase-cycle requirements.

Q34. 3-wire vs 4-wire SPI? Standard (4-wire) SPI uses separate MOSI and MISO lines for full-duplex operation; 3-wire SPI combines them into a single bidirectional data line (half-duplex), saving one pin at the cost of not being able to transmit and receive simultaneously.

Q35. SPI error handling? SPI has no built-in error detection/acknowledgment (unlike I2C’s ACK bit) — reliability typically relies on the higher-level protocol (checksums/CRC in the data payload, read-back verification, or status register polling on the slave) since the bus itself can’t signal a failed transfer.

Q36. Interrupt vs polling in SPI? Polling repeatedly checks a “transfer complete” status flag in a loop, simple but wastes CPU time; interrupt-driven SPI lets the CPU do other work and only services the transfer-complete (or DMA-complete) interrupt, more efficient for background/high-throughput transfers.

Q37. SPI clock speed limitations? Upper bound is set by the slowest device on the bus (slave’s max rated clock), signal integrity over the physical trace/cable length (SPI has no differential signaling, so it’s more susceptible to noise/reflections at high speed over longer wires), and the MCU’s own peripheral clock divider options.

Q38. Common SPI issues? Mismatched CPOL/CPHA mode between master and slave, incorrect or missing CS assertion/timing, MOSI/MISO lines swapped, clock speed exceeding the slave’s rating, and floating/unconnected CS on an unused slave causing bus contention.

Q39. SPI debugging steps? Verify CPOL/CPHA mode matches on both ends, use a logic analyzer to confirm CS is asserted for the full transaction and check bit-level timing against the datasheet, confirm MOSI/MISO aren’t swapped, and check clock frequency against the slave device’s maximum rating.

Q40. Real-world SPI use case? Common in automotive/embedded work for interfacing with external SPI flash/EEPROM (storing calibration data or bootloader images), high-speed sensors (accelerometers, pressure sensors), and displays — chosen over I2C when higher throughput or full-duplex operation is needed.

Advanced

Q41. SPI slave mode implementation on an MCU? The MCU’s SPI peripheral is configured as a slave (clock/CS driven externally by a master), shifting data in/out synchronized to the external SCLK; because the slave doesn’t control timing, it typically needs to have TX data pre-loaded before CS is asserted, or use interrupts/DMA to keep up in real time.

Q42. CS behavior between bytes — held vs toggled? Some devices expect CS to stay asserted for an entire multi-byte transaction (treating it as one logical command), while others require CS to toggle between each byte/command — this is device/datasheet-specific and a frequent source of “no response” bugs when assumed incorrectly.

Q43. Dummy bytes/clocks in SPI transactions? Some SPI devices (e.g., flash memory reads) require the master to clock out extra “dummy” bytes after a command before valid response data appears, giving the slave device internal processing time — the master must know and account for this turnaround gap per the datasheet.

Q44. CRC/checksums layered over SPI? Since SPI itself has no error detection, many sensor/flash protocols embed their own checksum or CRC within the data payload (e.g., an 8-bit CRC after sensor readings) that the master verifies in software to catch transmission errors the bus itself can’t detect.

Q45. SPI over longer distances/cables? SPI’s single-ended, non-differential signaling makes it susceptible to noise, reflections, and crosstalk over long traces/cables, especially at high clock speeds — generally kept to short on-board or short-cable runs; longer distances typically call for a differential bus (e.g., RS-485, CAN) or repeaters/buffers.

Q46. Multi-master SPI? Not natively supported by the SPI standard (no arbitration mechanism like I2C/CAN) — implementing multiple masters on one bus requires careful external coordination (e.g., a shared “bus grant” signal or software protocol) to avoid two masters driving MOSI/SCLK simultaneously.

Q47. Read-modify-write sequences over SPI? Common pattern for register access: send a “read register” command + address, clock out dummy bytes to receive the current value, then issue a “write register” command with the modified value — must be done atomically (CS held, or with interrupts disabled) to avoid another access interleaving.

Q48. SPI clock configuration mismatch handling? If a master talks to multiple slaves requiring different SPI modes (CPOL/CPHA) or clock speeds, the peripheral must be reconfigured between transactions to each slave’s specific requirements — a common bug is forgetting to reset the clock speed/mode before switching to a different slave.

Q49. Half-duplex 3-wire SPI nuances? With a single bidirectional data line, the master and slave must agree on direction per phase of the transaction (e.g., command out, then turn the line around for data in) — timing of the direction switch is critical and must match the device’s exact protocol to avoid bus contention.

Q50. SPI power-up/reset sequence expectations? Many SPI slave devices require a specific reset or

“dummy” CS toggle sequence after power-up before they respond correctly to real commands (e.g., to exit a power-down state or clear stale internal state) — check the datasheet’s power-on sequencing diagram if a device doesn’t respond right after boot.

Scenario-Based

Q51. Reading a full byte shifted by one bit. Classic symptom of a CPOL/CPHA (SPI mode) mismatch between master and slave — the sampling edge doesn’t align with when the slave has valid data ready, so bits appear shifted; fix by matching the SPI mode to the slave’s datasheet.

Q52. MISO reads all 0xFF or all 0x00. Usually means MISO is floating (no slave actually driving it — wrong CS, disconnected/misconfigured slave, or reading from an unselected slave) or a slave is stuck in reset/power-down; check CS assertion and slave power/wiring.

Q53. Multiple slaves respond simultaneously / bus contention. Indicates more than one CS line is asserted at once (a wiring or software bug), or an unused slave’s CS is left floating instead of pulled to its inactive level — always ensure exactly one CS is active at a time and unused CS lines are properly terminated.

Q54. Data corrupted only at high clock speeds. Points to signal integrity limits — trace/cable length exceeding what’s reliable at that frequency, insufficient drive strength, missing series termination, or exceeding the specific slave’s maximum rated clock; try reducing clock speed to confirm, then address wiring/termination.

Q55. Glitches on CS causing partial/garbled transactions. A noisy or bouncing CS line can cause the slave to see spurious assert/deassert edges mid-transaction, resetting its internal bit counter — check CS signal integrity on a scope and consider adding a small filter/debounce or verifying the GPIO driving CS has clean transitions.

Q56. DMA transfer produces wrong/incomplete data. Check DMA buffer size matches the expected transfer length exactly, confirm the DMA is triggered on the correct SPI event (TX empty / RX not empty), and verify CS is held low for the entire DMA-driven transfer if the slave expects one continuous transaction.

Q57. Device doesn’t respond at all after power-up. Check whether the device requires a specific reset/wake sequence, dummy clock cycles, or a settling delay after power-on before it accepts commands — many SPI ICs ignore the bus until an internal power-on reset/self-test completes.

Q58. Works on the bench but fails intermittently in the field. Consider marginal signal integrity under real-world noise/EMI/vibration (loose connector, cable flex), voltage supply dips affecting the slave’s timing, or a CS/clock line picking up noise near other switching circuitry — look at grounding, decoupling capacitors, and cable/connector robustness.

Q59. Mixed voltage-level SPI devices (3.3V master, 5V slave). Requires level shifting on all four SPI lines (SCLK, MOSI, MISO, CS) in the correct direction, since a 3.3V master’s outputs may not reliably register as “high” to a 5V-only slave and a 5V slave’s MISO output could exceed the 3.3V master’s absolute maximum input rating.

Q60. Debugging an unresponsive SPI slave. Use a logic analyzer to verify CS, SCLK, and MOSI are toggling as expected and match the exact command sequence in the datasheet; check SPI mode (CPOL/CPHA) and clock speed against the device’s spec; and confirm the slave’s power supply, reset, and any required pre-conditions before first access.

I2C

Q41. What is I2C? Inter-Integrated Circuit — a synchronous, half-duplex, multi-master/multi-slave serial protocol using just two open-drain wires (SDA for data, SCL for clock), where each slave device is addressed by a unique address rather than a dedicated chip-select line.

Q42. I2C signal lines? SDA (Serial Data, bidirectional) and **SCL** (Serial Clock, normally driven by the master) — both are open-drain and require external pull-up resistors to VDD since devices can only pull the lines low, never drive them high.

Q43. Master-slave communication? The master initiates every transaction by generating a START condition, then clocks out the target slave’s address plus a read/write bit; only the addressed slave responds (with an ACK and subsequent data), while all other slaves remain idle/listening.

Q44. START and STOP conditions? START: SDA transitions from high to low while SCL is high (signals the beginning of a transaction). **STOP**: SDA transitions from low to high while SCL is high (signals the end). Both are special conditions distinguishing framing bits from data bits (which only change while SCL is low).

Q45. ACK/NACK? After each 8-bit byte (address or data), the receiver pulls SDA low during the 9th clock pulse to acknowledge (ACK) successful reception; leaving SDA high (NACK) signals a problem (no such address, receiver full/busy, or the master ending a read after the last byte).

Q46. 7-bit vs 10-bit addressing? 7-bit addressing supports up to 128 device addresses (some reserved) and is by far the most common; 10-bit addressing extends the address space for busier systems with more devices, using a special reserved prefix in the first byte to signal a 10-bit address follows, at the cost of extra overhead and reduced compatibility with some legacy devices.

Q47. Clock stretching? A slave can hold SCL low after the master releases it, pausing the clock, when the slave needs more time to process data before the next bit/byte — the master must wait until it sees SCL actually go high before continuing, providing built-in flow control.

Q48. Multi-master arbitration? When multiple masters start a transaction simultaneously, each monitors SDA while it transmits; a master that intended to send a high bit but reads back a low bit (driven by another master) loses arbitration and immediately stops driving the bus, letting the other master(s) continue — similar in principle to CAN's non-destructive arbitration.

Q49. Pull-up resistors necessity? Since SDA/SCL are open-drain (devices can only pull low, not drive high), external pull-up resistors to VDD are mandatory to restore the lines to a high/idle state when no device is actively pulling them low; resistor value is chosen based on bus capacitance and desired speed (typically 2.2k-10k ohms).

Q50. I2C speed modes? Standard mode (100 kHz), Fast mode (400 kHz), Fast mode plus (1 MHz), High-speed mode (3.4 MHz), and Ultra-fast mode (5 MHz, push-pull, unidirectional) — higher speeds generally require smaller pull-up values and shorter/lower-capacitance bus wiring.

Q51. Repeated start? A START condition issued again without a preceding STOP, allowing the master to change direction (e.g., write a register address, then immediately read data) or address a different slave within a single continuous transaction — keeps the bus held by the current master, preventing another master from interjecting.

Q52. I2C vs SPI vs UART? I2C: 2 wires, addressed multi-device, half-duplex, moderate speed, built-in ACK. SPI: 4+ wires (extra per slave), full-duplex, fastest, no addressing/ACK. UART: 2 wires (point-to-point), asynchronous, no shared clock, simplest but limited to two devices per link.

Q53. Bus contention handling? Because SDA/SCL are open-drain, if two devices drive conflicting levels the line simply reads as low (wired-AND) rather than causing damage — this property underlies both clock stretching and multi-master arbitration, letting contention resolve safely without a bus fault.

Q54. Common I2C errors? Missing/incorrect pull-up resistors (weak or noisy signals), address conflicts between two slaves configured with the same address, bus lock-up from a slave holding SDA low indefinitely (often needs a manual bus-recovery clock sequence), and exceeding the maximum bus capacitance/length for the chosen speed.

Q55. I2C bus recovery? If a slave gets stuck holding SDA low (e.g., after a reset mid-transaction), the master can recover the bus by manually toggling SCL up to 9 times while watching SDA — this often lets the stuck slave finish shifting out its remaining bits and release SDA, followed by a manual STOP condition to reset bus state.

Q56. Timeout mechanism? Some I2C peripherals/drivers implement a software or hardware timeout so that if an expected ACK, clock-stretch release, or transaction completion doesn't occur within a set time, the driver aborts and resets the peripheral rather than hanging indefinitely — protects against a stuck bus/slave.

Q57. I2C register read/write? Typical pattern: master sends START + slave address + write bit, then the target register address byte, then either the data byte(s) to write, or issues a repeated START + slave address + read bit to then clock in the register's data — a two-phase "write pointer, then read/write data" sequence.

Q58. Interrupt handling in I2C? The I2C peripheral raises interrupts on events like address match (slave mode), byte received/transmitted, NACK received, or arbitration lost — the ISR typically manages the state machine of a multi-byte transaction and signals completion/error to the application layer.

Q59. Common I2C issues in embedded systems? Bus hang/lock-up after a mid-transaction reset,

address collisions when integrating third-party modules with fixed addresses, insufficient pull-up strength causing slow rise times at higher speeds, and clock-stretching slaves violating a master's assumed timeout.

Q60. I2C debugging steps? Check pull-up resistor values and confirm both lines idle high with a multimeter/scope, use a logic analyzer to inspect START/STOP framing, address byte, and ACK/NACK bits, verify no address conflicts among slaves, and check for a stuck-low SDA/SCL indicating a hung slave requiring bus recovery.

Advanced

Q61. SMBus vs I2C? SMBus (System Management Bus) is a stricter subset of I2C with defined timeouts, a fixed voltage range, mandatory clock-stretching timeout limits, and a packet error checking (PEC) byte for CRC — designed for reliability in power/system management applications, while I2C itself is more flexible but has fewer mandatory guarantees.

Q62. I2C multiplexers (e.g., TCA9548)? A multiplexer IC that allows a master to select one of several downstream I2C bus segments via register commands, letting multiple slave devices that share the same fixed address (e.g., identical sensor breakout boards) coexist on separate isolated channels behind a single master bus.

Q63. General call address? A reserved address (0x00) that all I2C slaves listening for it will respond to simultaneously, used for broadcast-style commands (e.g., a global reset command) rather than addressing one specific device.

Q64. Software (bit-banged) I2C vs hardware I2C peripheral? Bit-banged I2C toggles SDA/SCL as GPIOs under direct software control (flexible, works on any pins, but consumes CPU cycles and is timing-sensitive/less precise), while a hardware I2C peripheral handles bit-level timing, ACK generation, and clock stretching automatically in silicon, freeing the CPU and improving reliability/speed.

Q65. I2C repeaters / level shifters (bidirectional issue)? Because SDA/SCL are bidirectional open-drain lines, a simple unidirectional level shifter won't work — I2C-specific bidirectional level shifters/repeaters (or buffer ICs like the PCA9306) are needed to safely bridge two different voltage-domain I2C buses.

Q66. Dynamic/programmable slave addressing? Some I2C devices support an address-select pin or an in-band address-programming sequence, letting multiple identical devices be assigned distinct addresses on the same bus — avoids needing a multiplexer when the device itself supports configurable addressing.

Q67. I2C bus capacitance limits? The standard specifies a maximum total bus capacitance (400pF for standard/fast mode) from all device pins plus wiring; exceeding it slows the signal rise time beyond what the chosen speed and pull-up value can support, causing timing violations — long buses with many devices may need active buffering.

Q68. PMBus? A power-management-specific protocol layered on top of I2C/SMBus physical and framing rules, adding standardized commands for monitoring/controlling power supplies (voltage, current, temperature, fault status) — common in server/industrial power system designs.

Q69. I2C in multi-master automotive/embedded systems? Requires careful arbitration-aware design since two ECUs could attempt simultaneous transactions; many practical embedded designs sidestep this complexity by using a single master with a static bus topology, reserving true multi-master for cases where it's specifically needed.

Q70. Timeout and fault-tolerance strategies? Robust I2C drivers implement a maximum wait time for ACK/clock-stretch/transaction completion, an automatic bus-recovery sequence (manual SCL toggling) on timeout, and often a periodic bus-health check — critical in safety-relevant embedded systems where a single hung sensor shouldn't be able to freeze the whole application.

Scenario-Based

Q71. Bus permanently stuck low (SDA or SCL). Typically a slave got interrupted mid-byte (e.g., by a reset) and is still holding SDA low waiting to finish clocking out a bit — recover using the standard bus-recovery sequence (master manually pulses SCL up to 9 times while watching for SDA to release), followed by a manual STOP.

Q72. Device doesn't ACK its address. Check the address is correct (including the read/write bit and any 7-bit vs 8-bit address-notation confusion), confirm the device is actually powered and its address-

select pins are wired as expected, and verify no other device on the bus shares/conflicts with that address.

Q73. Intermittent NACKs under system load. Could indicate marginal pull-up strength that degrades under electrical noise from other switching activity, a slave that occasionally can't keep up (needs clock stretching but the master's timeout is too aggressive), or a shared power rail dipping under load and affecting logic thresholds.

Q74. Data corruption from two masters colliding. If arbitration isn't working correctly (e.g., a masters implementation bug, or hardware that doesn't properly monitor SDA while transmitting), simultaneous transactions can corrupt data instead of cleanly losing arbitration — verify each master's I2C peripheral correctly implements multi-master arbitration, or avoid true multi-master topologies if not required.

Q75. Driver times out waiting for clock stretch release. Check the slave's actual required processing time against the master driver's configured timeout, increase the timeout if the slave's stretching is expected/valid per its datasheet, or investigate why the slave is taking abnormally long (e.g., an internal fault or an overly slow internal operation).

Q76. Garbage data despite correct address/ACK. Often insufficient pull-up strength causing slow rise times that get misread at the configured speed (especially with a longer bus or more devices), electrical noise from nearby switching circuitry, or a marginal timing violation — check rise time on a scope against the I2C spec's requirement for the chosen speed mode.

Q77. Bus works fine alone, fails after adding a new module. Check for an address conflict with the newly added device, verify the new device doesn't add excessive bus capacitance requiring stronger pull-ups, and confirm its logic-level voltage is compatible with the existing bus voltage domain.

Q78. Hot-plugging a device causes the whole bus to hang. Plugging in a device while SCL happens to be low can leave a floating/mid-transaction wire that another device misinterprets, or cause a partial START condition — some designs add hot-swap-safe buffers, or require devices only be connected at power-off, to avoid this class of fault.

Q79. Reading a multi-byte register returns wrong values. Check whether the device's byte order (endianness) for multi-byte registers matches what the driver assumes, confirm a repeated START (not a STOP+new START) is used between the register-address write and the data read if the device requires it, and verify the correct number of bytes are being clocked out.

Q80. Debugging a suspected I2C bus fault. Use a logic analyzer to confirm clean START/STOP framing, correct address and ACK/NACK bits, and measure signal rise/fall times against the pull-up value and bus capacitance; check for address conflicts and confirm bus recovery resolves any stuck-low condition.

CAN

Basics

Q1. What is CAN? Controller Area Network — a robust, multi-master serial bus protocol designed for real-time communication between ECUs in vehicles/machinery, using differential signaling (CAN_H/CAN_L) for noise immunity, without needing a central host.

Q2. Why CAN? It offers high noise immunity (differential signaling), built-in error detection/handling, message-based (not address-based) multi-master arbitration without data collision, and reduces wiring compared to point-to-point connections — critical for harsh automotive electrical environments.

Q3. CAN architecture? A multi-master, message-broadcast bus where all nodes are connected in parallel on a twisted pair (CAN_H/CAN_L), terminated at both ends with 120-ohm resistors. Every node "hears" every message; each node's CAN controller + transceiver handles bit-level transmission, arbitration, and error detection, passing valid frames up to the application via filters.

Q4. Standard frame? CAN 2.0A frame with an 11-bit identifier, allowing up to 2048 unique IDs, used for basic CAN networks — structure: SOF, 11-bit ID, RTR, control field, up to 8 data bytes, CRC, ACK, EOF.

Q5. Extended frame? CAN 2.0B frame with a 29-bit identifier (11-bit base ID + 18-bit extension), allowing over 536 million unique IDs, distinguished from standard frames by the IDE bit being set to 1.

Q6. CAN FD? CAN with Flexible Data-rate — an extension supporting up to 64 data bytes per frame (vs 8

in classic CAN) and a higher bit rate during the data phase, while keeping the arbitration phase at the classic rate for backward compatibility. Improves throughput for modern ECU data needs.

Q7. Arbitration? The non-destructive, bitwise contention-resolution method where multiple nodes transmitting simultaneously compare their identifier bit-by-bit on the bus; a node sending a recessive (1) bit while another sends dominant (0) loses arbitration and backs off, so the frame with the lowest ID (highest priority) wins without any data loss or retransmission delay.

Q8. Bit stuffing? After 5 consecutive identical bits, the transmitter automatically inserts one bit of the opposite polarity to ensure enough transitions for receiver clock synchronization (NRZ encoding needs edges); receivers automatically remove stuffed bits. Violation of this rule is detected as a stuff error.

Q9. CRC? Cyclic Redundancy Check — a 15-bit checksum computed over the frame contents and appended in the CRC field; each receiver recomputes the CRC and compares it, flagging a CRC error if there's a mismatch, ensuring data integrity.

Q10. ACK? Acknowledge bit — the transmitter sends this bit recessive, and any receiver that correctly received the frame (valid CRC) overwrites it with a dominant bit during that slot, confirming at least one node received the message correctly. A missing ACK (still recessive) indicates no receiver acknowledged it.

Advanced

Q11. Error frame? A frame transmitted by a node upon detecting a bus error (bit, stuff, CRC, form, or ACK error), consisting of an error flag (6 dominant bits, violating stuffing rules to alert all nodes) followed by an error delimiter, causing all nodes to discard the current message.

Q12. Overload frame? A frame a node transmits to request extra delay between messages, e.g., when it needs more time to process the previous frame before the next one arrives, or in response to a dominant bit detected during intermission.

Q13. Remote frame? A frame with the RTR (Remote Transmission Request) bit set and no data payload, used by a node to request data from another node that owns a particular identifier — the addressed node responds with the corresponding data frame.

Q14. Bus Off? A fault-confinement state entered when a node's Transmit Error Counter (TEC) exceeds 255, in which the node electrically disconnects itself from the bus and stops all transmission/reception until it's reset (typically via 128 occurrences of 11 consecutive recessive bits, or a controller reset) — protects the healthy bus from a persistently faulty node.

Q15. Error Passive? A fault-confinement state entered when either the Transmit Error Counter (TEC) or Receive Error Counter (REC) exceeds 127; the node can still transmit/receive but must send passive error flags (recessive, less disruptive) instead of active ones, and must wait extra time before transmitting again.

Q16. Error Active? The normal, healthy fault-confinement state (both TEC and REC below 128) where a node can transmit active error flags (dominant bits) that actively disrupt the bus to signal an error to all nodes — the default state at power-up.

Q17. Baud rate? The bit transmission rate of the CAN bus (e.g., 125 kbps, 250 kbps, 500 kbps, 1 Mbps), determined by the bit time configuration (propagation segment, phase segments, sync segment) and must be identical for all nodes on the bus.

Q18. Sample point? The specific point within a bit time (expressed as a percentage) at which each node samples the bus level to determine the bit's value, typically placed around 75-87.5% into the bit time to allow signal propagation and settling time before sampling.

Q19. Termination resistor? A 120-ohm resistor placed at each of the two physical ends of the CAN bus to match the cable's characteristic impedance, preventing signal reflections that would corrupt data — a healthy standard CAN bus measures about 60 ohms (two 120-ohm resistors in parallel) between CAN_H and CAN_L.

Q20. CAN physical layer? Defined by ISO 11898-2 (high-speed CAN) — uses a twisted-pair differential bus (CAN_H, CAN_L); dominant bit (logical 0) drives CAN_H high/CAN_L low (larger voltage difference), recessive bit (logical 1) lets both lines float near a common voltage (smaller/no difference), giving strong noise immunity.

Practical

Q21. Explain one CAN frame. A standard data frame consists of: Start of Frame (1 dominant bit), 11-bit Identifier, RTR bit, Control field (IDE, reserved bit, 4-bit DLC specifying 0-8 data bytes), Data field (0-8 bytes), 15-bit CRC + CRC delimiter, 2-bit ACK slot + delimiter, 7-bit End of Frame, and Intermission (3 bits) before the next frame.

Q22. How did you debug CAN? Typical workflow: connect a CAN analyzer/PCAN-USB to tap the bus, capture live traffic to check for the expected IDs/periodicity/data, verify bus loading and error counters, check termination resistance and bus voltage levels with a multimeter/scope if frames are corrupted, and correlate suspect frames against the DBC file to decode signals and pinpoint the faulty ECU or wiring issue.

Q23. PCAN? PEAK-System's PCAN-USB — a common USB-to-CAN interface adapter used with PC software (PCAN-View, or custom tools) to monitor, send, and log CAN traffic during development and diagnostics; it's a hardware device Sriram has used for flashing/diagnostic tool development.

Q24. BusMaster? A free, open-source CAN bus analysis tool (originally from Robert Bosch) used to monitor, transmit, log, and simulate CAN traffic, supporting DBC-based signal decoding and scripting for test automation.

Q25. CANalyzer? A Vector Informatik tool for analyzing, monitoring, and testing CAN (and other automotive bus) traffic — widely used in the industry for advanced tracing, DBC-based decoding, diagnostics (UDS) tracing, and network simulation.

Q26. CANoe? Vector's more advanced ECU development and test tool, extending CANalyzer with simulation of complete networks/ECUs, automated test execution (CAPL scripting), residual bus simulation, and diagnostic (UDS/OBD) testing — commonly used in Tier-1 validation environments.

Q27. Acceptance filter? Hardware in the CAN controller that compares incoming message IDs against configured filter/mask registers, allowing only messages matching the criteria to be stored in the receive buffer/FIFO and interrupt the CPU, reducing software overhead from irrelevant traffic.

Q28. Mask? Used together with a filter ID — bits set to 1 in the mask mean "this bit must match the filter exactly," while bits set to 0 mean "don't care." This allows filtering a whole range of related IDs with one filter/mask pair instead of listing every ID individually.

Q29. CAN IDs? The identifier field of a CAN frame that both indicates message content/type and determines bus priority during arbitration — lower numeric ID value means higher priority (more dominant bits early in arbitration win the bus).

Q30. Priority? Determined purely by the identifier value during arbitration — the frame with the numerically lowest ID (most dominant leading bits) always wins bus access first, ensuring high-priority messages (e.g., safety-critical ABS/brake signals) are never delayed by lower-priority ones.

Scenario-Based

Q31. CAN Bus Off. Indicates a node's Transmit Error Counter exceeded 255 due to sustained transmission errors — investigate wiring/short circuits, missing termination, a faulty transceiver, ground offset issues, or another node stuck dominant; check error counters and error frame history on the analyzer, then reset the node's CAN controller once the root cause is fixed.

Q32. Missing ACK. Means no other node acknowledged receipt — possible causes: the transmitting node is isolated/disconnected from the rest of the bus, no other node is powered/listening, a receiver's acceptance filter excludes that ID, or a wiring fault between transmitter and receivers. Check bus connectivity and confirm at least one other live node is present and correctly filtered.

Q33. CRC Error. The receiver's computed CRC didn't match the transmitted CRC field, indicating data corruption in transit — often caused by electrical noise, poor termination/reflections, marginal signal levels, or a bit error during transmission. Check bus signal integrity (scope), termination, and shielding/grounding.

Q34. Stuff Error. Detected when 6 consecutive identical bits appear on the bus, violating the bit-stuffing rule — usually indicates a bit-level corruption (noise, glitch) or a node stuck dominant/recessive on the line.

Q35. Form Error. Occurs when a fixed-format bit field (e.g., CRC delimiter, ACK delimiter, EOF) doesn't have its expected recessive value — often points to timing/synchronization issues or a malfunctioning transceiver/controller.

Q36. Bit Error. Detected when a transmitting node monitors the bus and reads back a bit different from

what it sent (outside the arbitration/ACK phases where this is expected) — indicates a bus-level electrical problem: short circuit, wiring fault, or bus contention.

Q37. No response. For a diagnostic/UDS request with no reply — check whether the request was sent on the correct CAN ID/channel, whether the target ECU is powered and its filters are configured for that ID, session/security-access prerequisites, and use the analyzer to confirm the request actually appeared on the bus.

Q38. Wrong ID. A message appears with an unexpected identifier — check the DBC/ID mapping for a possible mismatch between what firmware sends and what's expected, verify no ID collision with another ECU, and confirm filter/mask configuration on the receiving node.

Q39. Lost arbitration. A node loses arbitration when it sends a recessive bit but reads back a dominant bit during the arbitration field — this is normal bus behavior (a higher-priority message is winning), not an error; the losing node automatically stops transmitting and retries after the bus is free.

Q40. Debugging steps. General approach: (1) reproduce the issue while capturing bus traffic on an analyzer, (2) check error counters/frames and bus load, (3) verify physical layer — termination, voltage levels, wiring/connectors, ground — with a multimeter/scope, (4) cross-check captured frames against the DBC for correct IDs/DLC/signal values, (5) isolate nodes one at a time if a faulty ECU is suspected, and (6) correlate timing with software logs/traces on the ECU under test.

CAN — Additional Advanced & Scenario Questions

Q41. Listen-only mode? A CAN controller mode where the node can receive and monitor bus traffic (useful for tools/analyzers or a node still initializing) but never transmits — including never sending dominant ACK or error frames — so it can't disturb the bus even if it detects an "error."

Q42. CAN transceiver standby/sleep mode? Many CAN transceivers support a low-power standby mode (bus transmitter/receiver largely powered down) that can be woken by a specific wake-up pattern on the bus, used for network-wide low-power modes in vehicles (e.g., ignition off) with wake-on-CAN-activity support.

Q43. ISO-TP (ISO 15765-2)? A transport-layer protocol on top of CAN that segments messages larger than 8 bytes (classic CAN's payload limit) into multiple frames — single frame, first frame, consecutive frames, and flow-control frames — enabling multi-frame UDS diagnostic messages to be sent over standard CAN.

Q44. J1939 vs plain CAN? J1939 is a higher-layer protocol (mostly for heavy-duty vehicles) built on 29-bit extended CAN IDs, defining standardized Parameter Group Numbers (PGNs), multi-packet transport (similar in spirit to ISO-TP), and network management — versus "plain" CAN, which only defines the bus/frame mechanics with application meaning left undefined.

Q45. DBC file? A CAN database file format (Vector's format, widely adopted) that defines message IDs, DLC, and the bit position/scaling/offset/unit of each signal packed within a message's data bytes — used by analyzer tools (CANoe, BusMaster) to decode raw CAN traffic into human-readable signal values.

Q46. Gateway ECU / CAN bus routing? An ECU that bridges multiple physically separate CAN buses (e.g., powertrain bus and body bus), selectively forwarding relevant messages between them — used to segment traffic for bandwidth/security reasons while still sharing necessary signals across domains.

Q47. Bus load calculation? The percentage of total bus bandwidth consumed by all messages' combined transmission time at the configured baud rate; high bus load (commonly kept under ~70-80% in design) risks priority inversion for lower-priority messages and reduced margin for error recovery/retransmission.

Q48. Wake-up frame / partial networking? Some CAN networks support "partial networking," where certain nodes can sleep while others remain active, and a specific wake-up frame (or dedicated wake pattern) brings sleeping nodes back online only when relevant traffic requires it, saving vehicle standby power.

Q49. CAN with redundant/dual bus architecture. Used in safety-critical systems where two independent CAN buses (or a bus plus a backup path) carry duplicate or complementary safety data, so a single bus fault (short, Bus Off) doesn't cause total loss of the safety function — relevant to ABS/ESC systems where signal availability is critical.

Q50. Scenario — Intermittent frame drops only under heavy bus load. Suggests bus load is approaching saturation, causing lower-priority messages to be repeatedly delayed or occasionally missed by receivers with small buffers/FIFOs — check bus load percentage, review message periodicity/priority assignment, and consider whether some traffic can be reduced, batched, or moved to a separate bus.